



RAJALAKSHMI
ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

CS23432 – Software Construction

(REGULATION 2023)

RAJALAKSHMI ENGINEERING COLLEGE
Thandalam, Chennai-602015

Name: Sukirthan Udhayakumar

Register No: 240701546

Year / Branch / Section: II / CSE

Semester: IV

Academic Year: 2025 - 2026

INDEX

S.No.	Date	Title
1.	07-02-2026	Azure DevOps Environment Setup
2.	14-02-2026	Azure Project Setup & User Stories
3.	21-02-2026	Epics, Features, User Stories Creation
4.	28-02-2026	Sprint Planning using Azure Boards
5.	07-03-2026	Poker Estimation using Azure Boards.
6.	14-03-2026	Designing Class and Sequence Diagrams for Product Category Management System.
7.	04-04-2026	Designing Architectural and ER Diagrams for Product Category Management System.
8.	11-04-2026	Testing – Test Suites and Test Cases in Azure.
9.	18-04-2026	Load Testing and CI/CD Pipeline Setup.
10.	25-04-2026	GitHub: Project Structure & Naming Conventions.

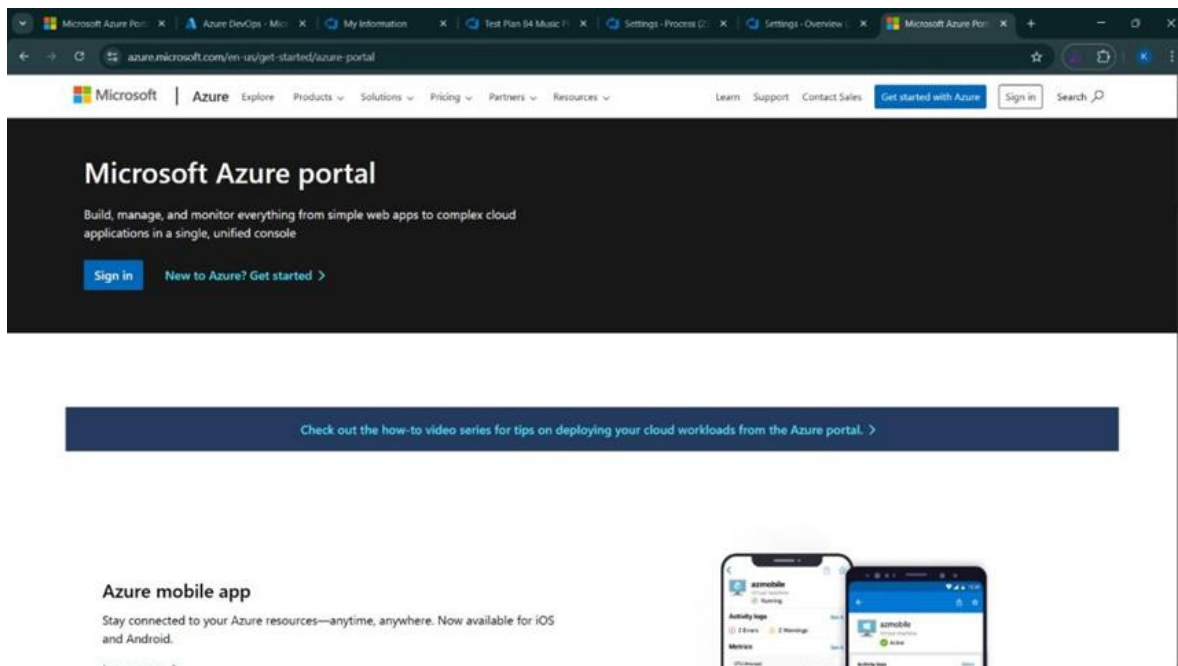
EXP NO: 01	AZURE DEVOPS ENVIRONMENT SETUP
------------	---------------------------------------

Aim:

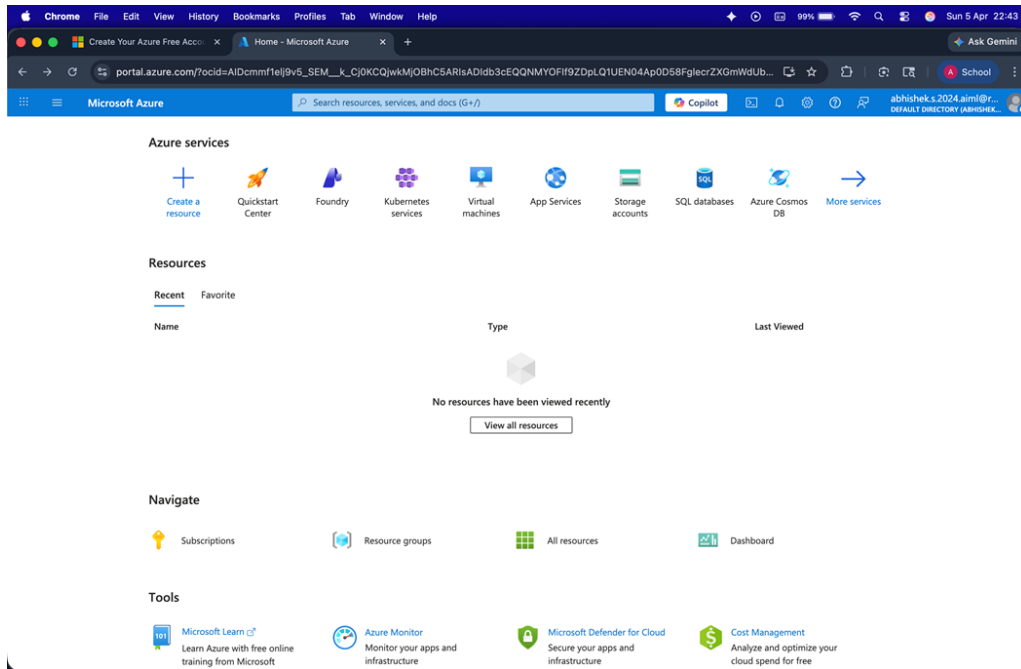
To set up and access the Azure Devops environment by creating an account and workspace for project management.

INSTALLATION

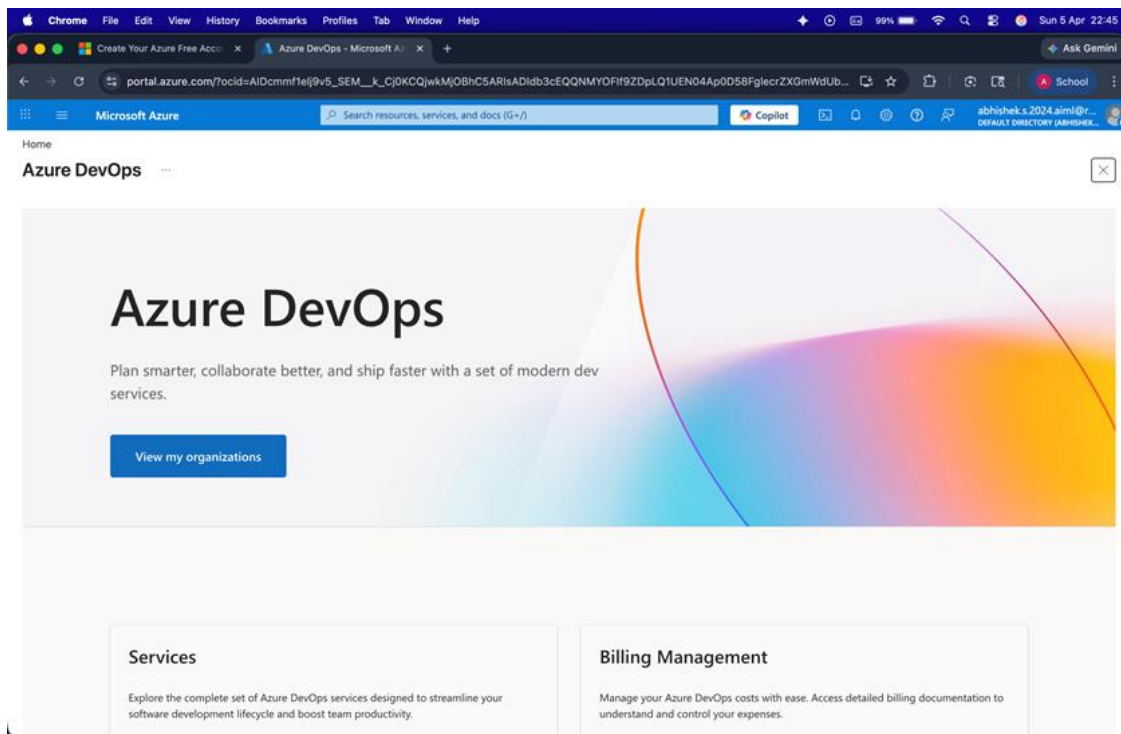
1. Open your web browser and go to: <https://dev.azure.com>
Sign in using your Microsoft account credentials.
If you do not have an account, create one using Sign Up option. .



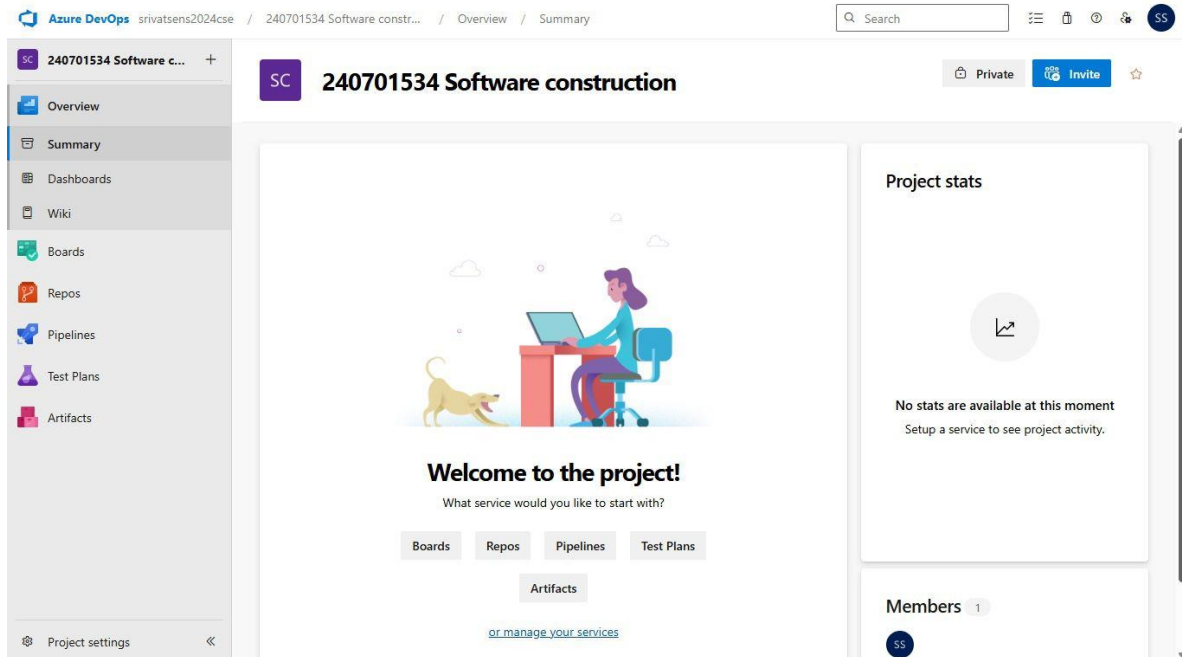
2. Azure DevOps home page



3. Open DevOps environment in the Azure platform by typing **Azure DevOps Organizations** in the search bar.



4. Click on **New Project**, create a new organization/project, and access the Azure DevOps dashboard successfully..



Result:

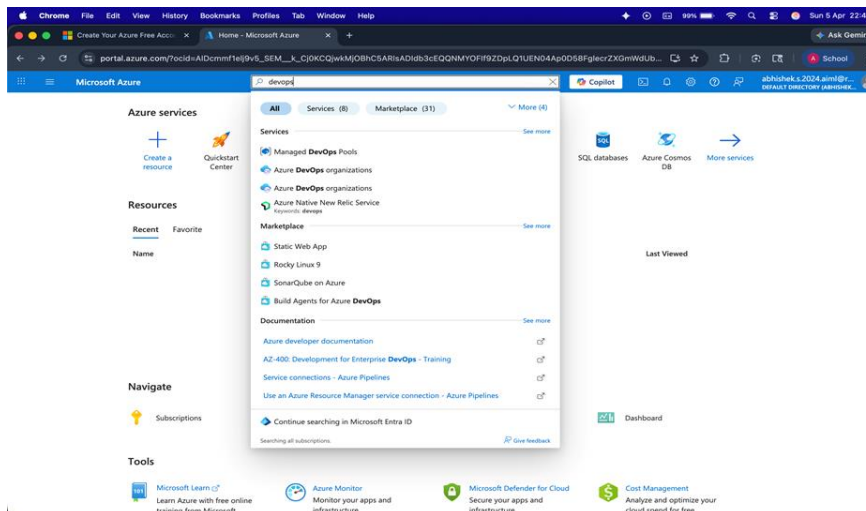
Successfully accessed the Azure environment and created a new workspace/site for project management.

EXP NO: 02	AZURE PROJECT SETUP AND USER STORY MANAGEMENT
------------	--

Aim:

To set up an Azure DevOps project for efficient collaboration and agile work management.

- Create an Azure DevOps Account
- Create the First Project in Your Workspace
 - a. After setup, create your first project.
 - b. Click **New Project**
 - c. Enter details:
 Name: Product Category Management System
 Description: Category handling system
 Template: Scrum
 Access: Private/Open
 - d. Click Create
- Open project dashboard
- Project dashboard
- To manage user stories:
 - a. Go to **Boards** → **Work Items**
 - b. Click **New Work Item** → **User Story**
 - c. Enter details



Result: Successfully created an Azure DevOps project with user story management

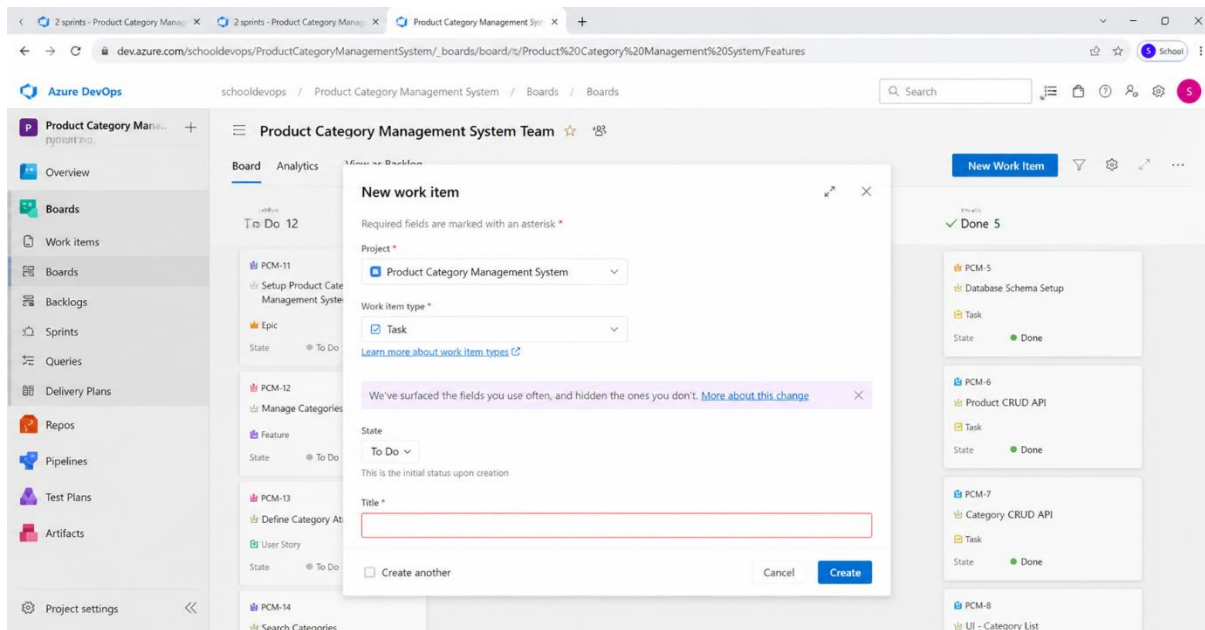
EXP NO: 03

SETTING UP EPICS, TASKS, AND USER STORIES FOR PROJECT PLANNING

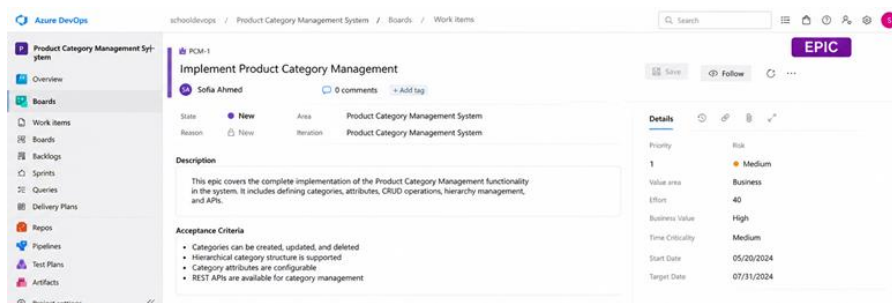
Aim:

To learn about how to create epics, tasks, user stories, and backlogs for your assigned project.

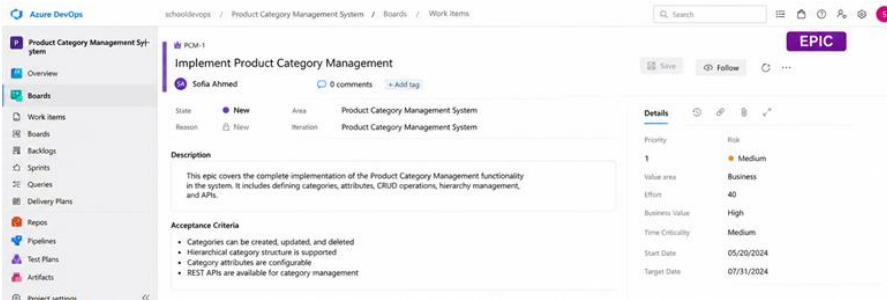
Create Epic, User Stories, Task



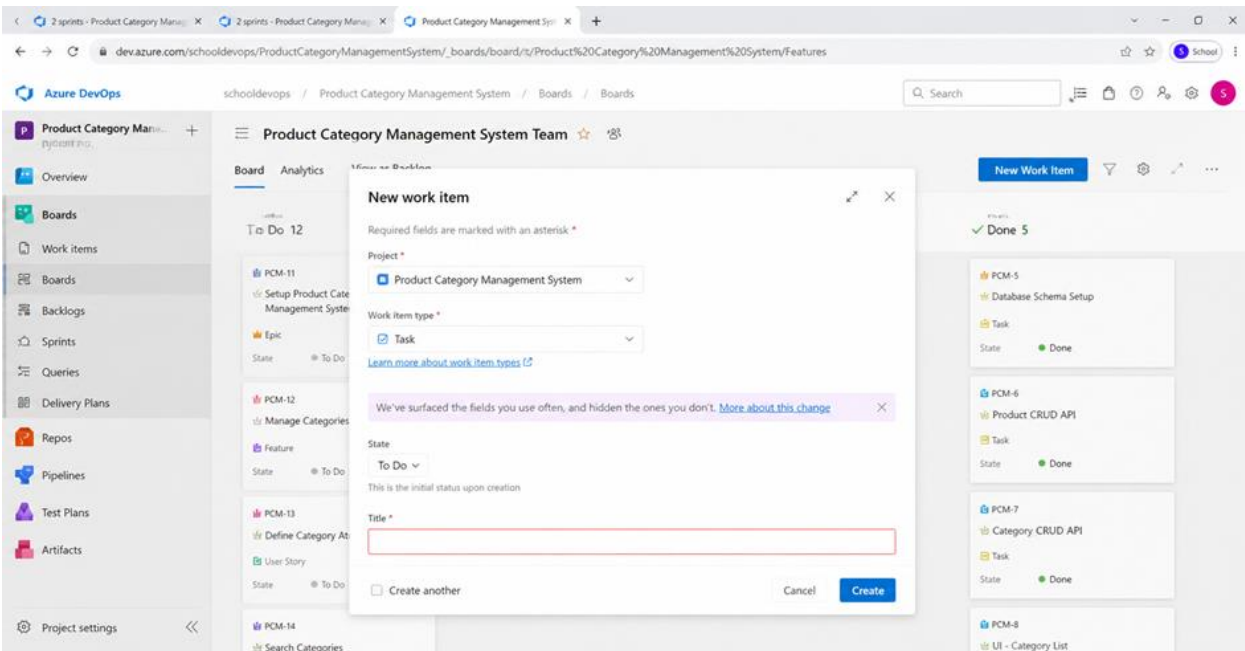
1.Fill in Epics



2.Fill in Features



3.Fill in User Story Details



Result:

Thus, the creation of epics, features, user story and task has been created successfully.

EXP NO: 04

SPRINT PLANNING IN AZURE

Aim:

To assign user story to specific sprint for the Product Category Management System

Sprint Planning

1. Open **Boards** → **Sprints**
2. Create:
 - Sprint 1
 - Sprint 2
3. Drag user stories into sprint

Sprint 1

The screenshot displays the Azure DevOps Sprints interface. On the left, a sidebar contains navigation options: Overview, Boards, Work items, Boards, Backlogs, Sprints (selected), Queries, Delivery Plans, Analytics views, Repos, Pipelines, Test Plans, and Artifacts. At the bottom of the sidebar is 'Project settings'. The main area is titled 'Taskboard' and includes tabs for Backlog, Capacity, and Analytics. It shows a filter for 'sprint-2' and 'Person: All'. A date range '6 April - 7 April' with '2 work days remaining' is displayed. Below this, a table with columns 'New', 'Active', 'Resolved', and 'Closed' lists two sprints: '19 sprint-1' and '18 sprint-2'. Each sprint has a 'New' item and an 'Unassigned' item. A 'Collapse all' button is visible at the top left of the table.

Sprint 2

The screenshot displays a Jira board for the team 'ai healthcare chat bot Team'. The board is currently in 'Taskboard' view, showing two sprints: 'sprint-1' and 'sprint-2'. Each sprint contains one work item, 'New', which is currently 'Unassigned'. The board is organized into columns: 'New', 'Active', 'Resolved', and 'Closed'. The left sidebar provides navigation options, including 'Overview', 'Boards', 'Work items', 'Backlogs', 'Sprints', 'Queries', 'Delivery Plans', 'Analytics views', 'Repos', 'Pipelines', 'Test Plans', and 'Artifacts'. The top right corner shows the date range '6 April - 7 April' and '2 work days remaining'.

	New	Active	Resolved	Closed
sprint-1	19 sprint-1 New Unassigned			
sprint-2	18 sprint-2 New Unassigned			

Result:

The Sprints are created for the Product Category Management System.

EXP NO: 05	POKER ESTIMATION USING AZURE
-------------------	-------------------------------------

Aim:

Create Poker Estimation for the user stories – Product Category Management System.

Poker Estimation

Points	Meaning
1	Trivial – minor UI text change, config tweak
2	Simple – single form field, small validation rule
3	Small – single API endpoint, basic DB query
5	Medium – feature with frontend + backend + DB
8	Large – complex feature with multiple integrations
13	Very Large – cross-system feature, high uncertainty
21	Epic – needs breaking down before sprint

Procedure:

- Assign points:
 - Add Category → 3
 - Delete Category → 2
 - Search → 1

Sprint	Total SP	Duration	Velocity	Focus
Sprint 1	39 SP	2 weeks	~20 SP/dev/sprint	Core auth + chatbot + symptom analysis
Sprint 2	62 SP	2 weeks	~31 SP/dev/sprint	Voice, maps, admin, monitoring, CI/CD

Result:

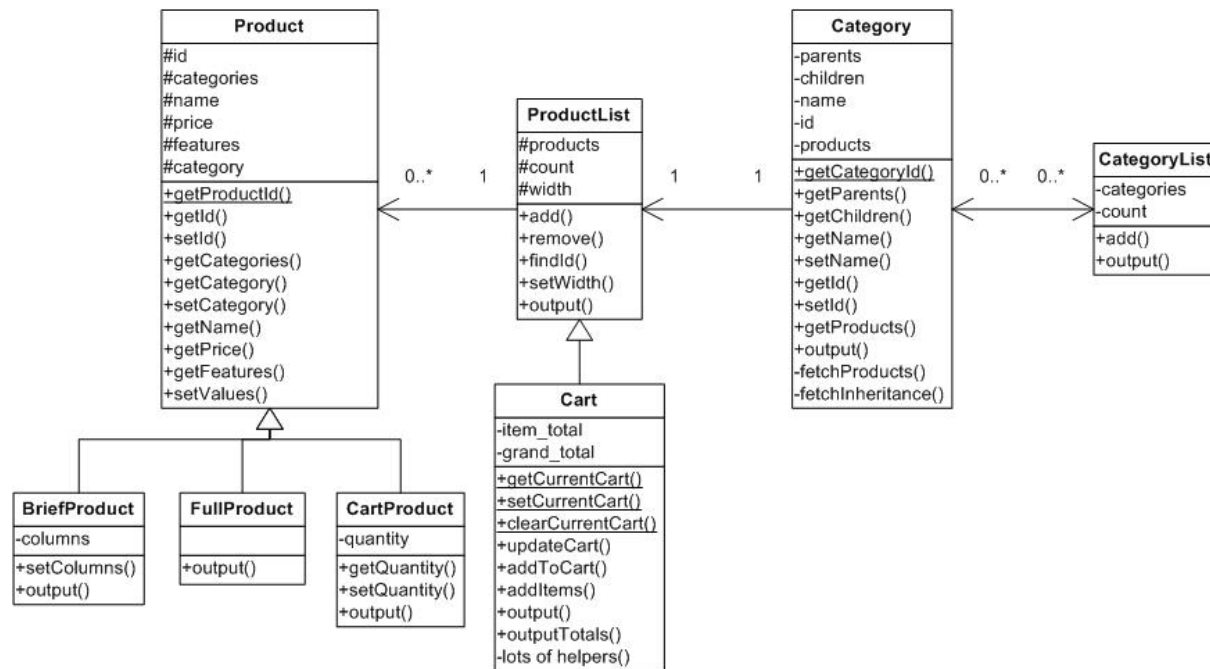
The Estimation/Story Points is created for the project using Poker Estimation.

EXP NO: 06	DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR PRODUCT CATEGORY MANAGEMENT SYSTEM
------------	---

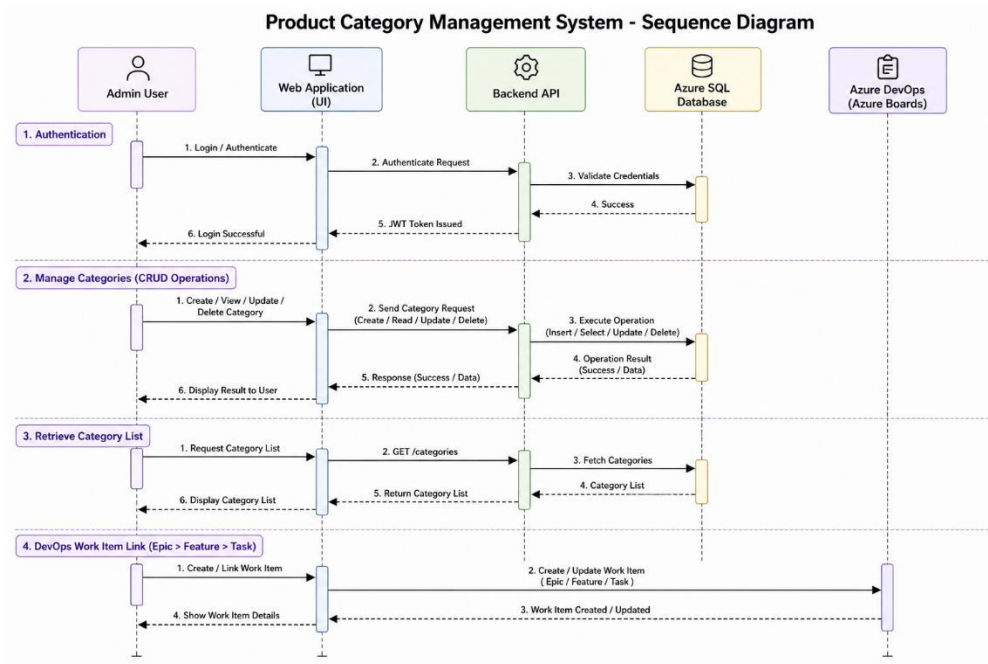
Aim:

To Design a Class Diagram and Sequence Diagram for the given Product Category Management System.

6A. Class Diagram



6B. Sequence Diagram



Result: The Class Diagram and Sequence Diagram is designed Successfully for the Product Category Management System

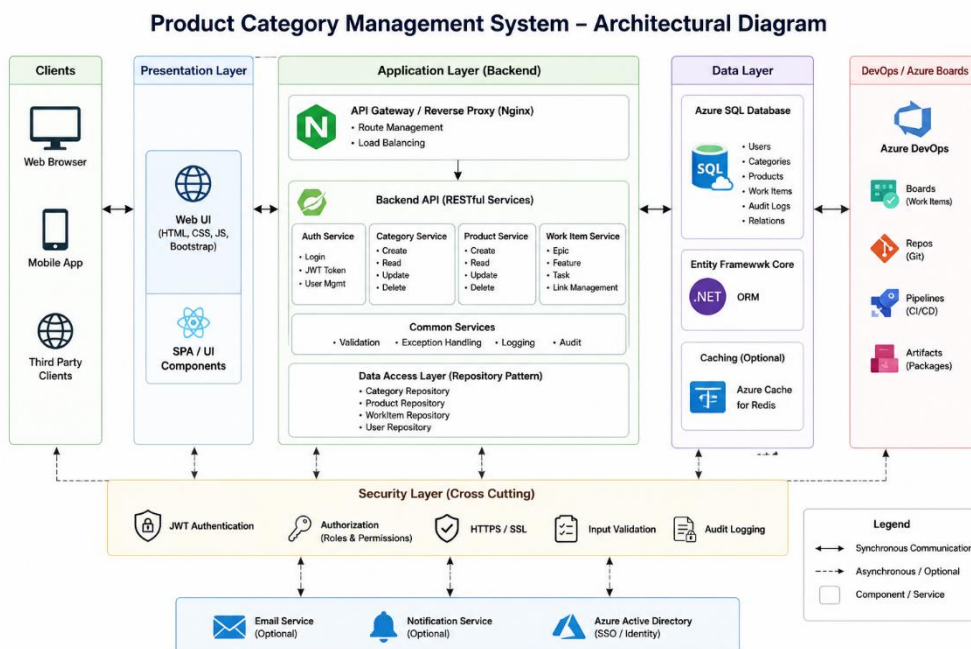
EXP NO: 07

DESIGNING ARCHITECTURAL AND ER DIAGRAMS FOR PRODUCT TRACKING SYSTEM

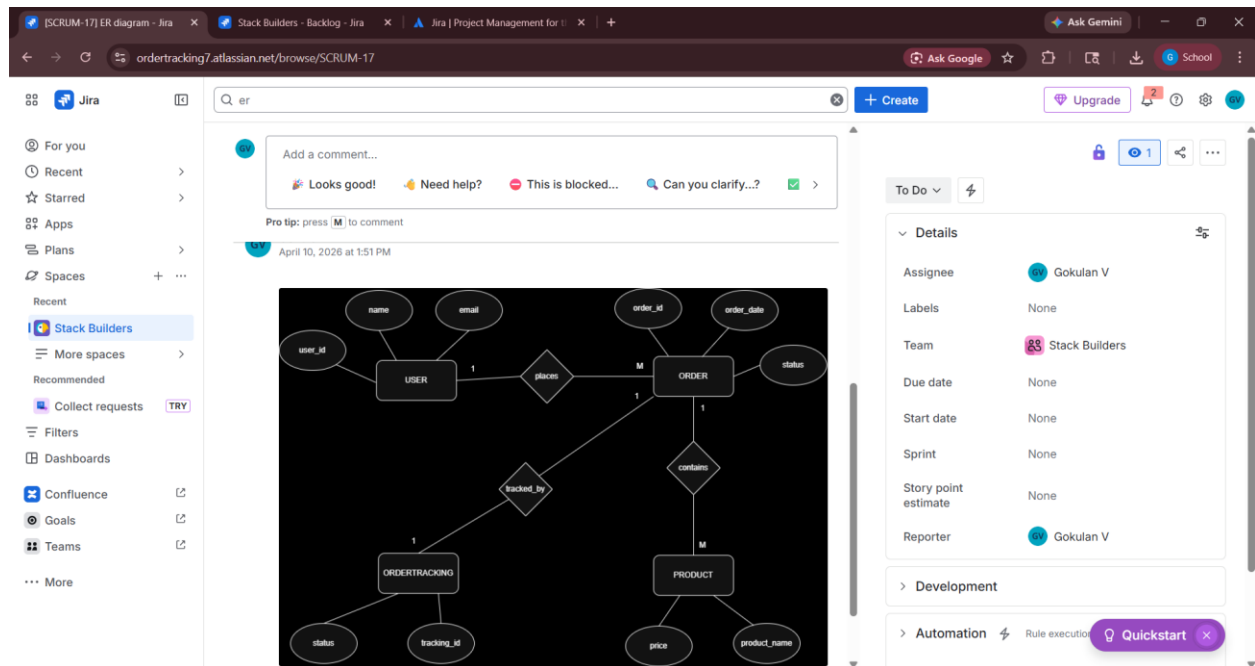
Aim:

To Design an Architectural Diagram and ER Diagram for the given Project.

7 A. Architectural Diagram



7 B. ER Diagram



Result

The Architecture Diagram and ER Diagram is designed Successfully for the Product Category Management System

EXP NO: 08	TESTING – TEST SUITES AND TEST CASES IN AZURE
-------------------	--

Aim:

To create Test plans and Test Cases, and write two test cases for atleast five user stories showcasing happy path and error scenarios in the Azure platform.

Test Planning and Test Case

Test Case Design Procedure

1. Understand Core Features of the Application

- User Signup & Login
- Place New Product
- Track Product Status
- View Product Details
- Admin Product Management

2. Define User Interactions

- Each test case simulates a real user behaviour

3. Design Happy Path Test Cases

- Focused on validating that all features function as expected under normal conditions.

4. Design Error Path Test Cases

- Simulate negative or unexpected scenarios to test robustness and error handling.

5. Break Down Steps and Expected Results

- Each test case contains step-by-step actions and a corresponding expected outcome.
- Ensures clarity for both testers and automation scripts.

6. Use Clear Naming and IDs

- Test cases are named clearly (e.g., TC01 – Successful Login, TC04 – Invalid Product Tracking).
- Helps in quick identification and linking to user stories.

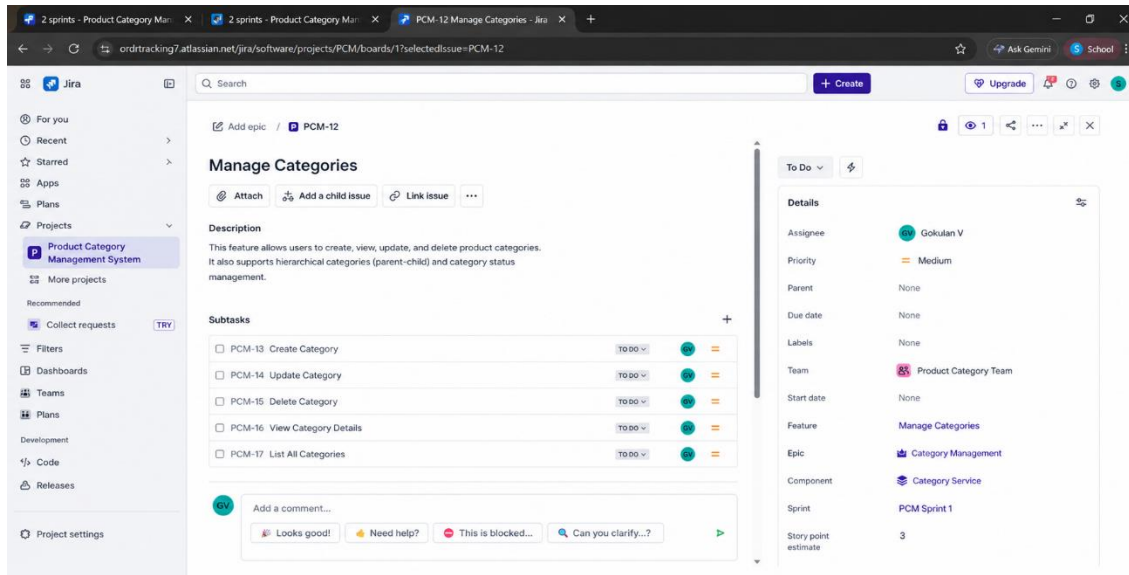
7. Separate Test Suites

- Grouped test cases based on functionality (e.g., Login Management, Product Management,
- Improves organization and test execution flow in Azure.

8. Prioritize and Review

- Critical user actions are marked high-priority.
- Reviewed for completeness and traceability against project requirements.

1. Test suite



2. Test case

Give two test cases for at least five user stories showcasing the happy path and error scenarios in Azure platform.

Product Tracking System – Test Plans

USER STORIES

- 🔪 As a user, I want to sign up and log in securely so that I can access the system (ID: US01).
- 🔪 As a user, I want to place a new product easily (ID: US02).
- 🔪 As a user, I want to track my product status using Product ID (ID: US03).
- 🔪 As a user, I want to view complete product details (ID: US04).
- 🔪 As an admin, I want to view and manage all customer products (ID: US05).

Test Suites

Test Suit: TS01 - User Login

1. TC01 – Successful Login

- **Action:**

- Open login page.
- Enter valid email and password.
- Click "Login".
- **Expected Results:**
 - Login successful.
 - User redirected to dashboards.
- **Type:** Happy Path

2. TC02 – Invalid Login

- **Action:**
 - Open Login page.
 - Enter invalid email or password.
 - Click on "Login".
- **Expected Results:**
 - Error message “Invalid credentials” displayed.
- **Type:** Error Path

Test Suit: TS02 – Product Creation

1. TC03 – Valid Product Creation

- **Action:**
 - Login successfully.
 - Enter product name and quantity.
- **Expected Results:**
 - Product created successfully.
 - Product ID generated.
- **Type:** Happy Path

2. TC04 – Empty Field Product Creation

- **Action:**
 - Open product form.
 - Leave required fields blank.
 - Click place product.
- **Expected Results:**
 - Validation message displayed.
- **Type:** Error Path

Test Suit: TS03 – Product Tracking

1. TC05 – Valid Product Tracking

- **Action:**
 - Enter valid Product ID.

- Click Track Product.
- **Expected Results:**
 - Current product status displayed.
- **Type:** Happy Path

2. TC06 – Invalid Product Tracking

- **Action:**
 - Enter wrong Product ID.
 - Click Track Product.
- **Expected Results:**
 - Message “Product not found” displayed.
- **Type:** Error Path

Test Suit: TS04 – View Product Details

1. TC07 – View Existing Product Details

- **Action:**
 - Enter Valid Product ID.
 - Click view details.
- **Expected Results:**
 - Customer name, product, quantity, date and status displayed.
- **Type:** Happy Path

2. TC08 – View Non Existing Product

- **Action:**
 - Enter Invalid product ID.
 - Click "View Details".
- **Expected Results:**
 - Message “Product not found” displayed.
- **Type:** Error Path

Test Suit: TS05 – Admin Product Management

1. TC09 – View All Products

- **Action:**
 - Admin Logs in.
 - Open Manage Products page.
- **Expected Results:**
 - All products displayed in list format.
- **Type:** Happy Path

2. TC10 – Unauthorized Access

- **Action:**
 - Normal user tries to access admin page.
- **Expected Results:**
 - Access denied message displayed.
- **Type:** Error Path

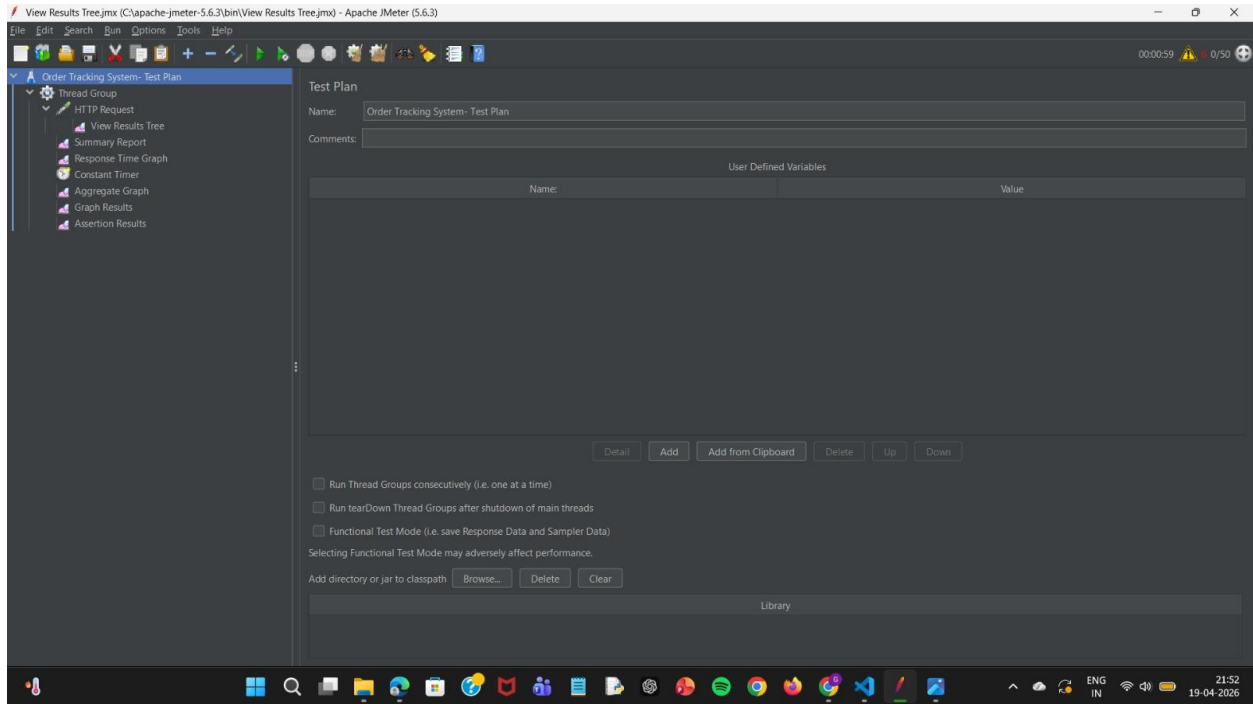
3. Test Cases

The screenshot shows the Jira web interface. The left sidebar contains navigation options like 'For you', 'Recent', 'Starred', 'Apps', 'Plans', 'Projects', 'Product Category Management System', 'More projects', 'Recommended', 'Collect requests', 'Filters', 'Dashboards', 'Teams', 'Plans', 'Development', 'Code', 'Releases', and 'Project settings'. The main content area is titled 'Manage Categories' for issue PCM-12. It includes a description: 'This feature allows users to create, view, update, and delete product categories. It also supports hierarchical categories (parent-child) and category status management.' Below the description is a 'Subtasks' section with a list of tasks: PCM-13 Create Category, PCM-14 Update Category, PCM-15 Delete Category, PCM-16 View Category Details, and PCM-17 List All Categories. Each task has a 'TO DO' status and a progress indicator. On the right, the 'Details' panel shows fields like Assignee (Gokuln V), Priority (Medium), Parent (None), Due date (None), Labels (None), Team (Product Category Team), Start date (None), Feature (Manage Categories), Epic (Category Management), Component (Category Service), Sprint (PCM Sprint 1), and Story point estimate (3). At the bottom, there is a comment section with a text input and buttons for 'Looks good!', 'Need help?', 'This is blocked...', and 'Can you clarify...?'.

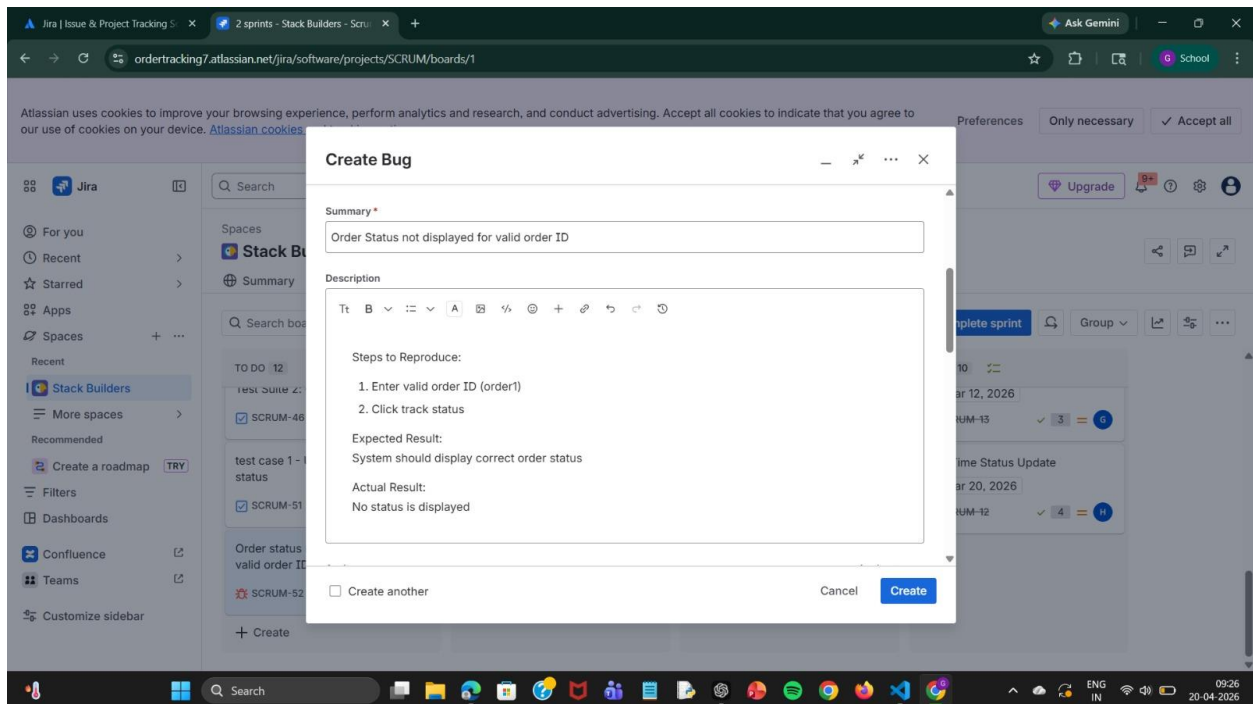
4. Installation of test (Apache JMeter)

The screenshot shows the Apache JMeter download page at jmeter.apache.org/download_jmeter.cgi. The page has a sidebar with links for 'About' (Overview, License), 'Download' (Download Releases, Release Notes), 'Documentation' (Get Started, User Manual, Best Practices, Component Reference, Functions Reference, Properties Reference, Change History, Javadoc, JMeter Wiki, FAQ (Wiki)), 'Tutorials' (Distributed Testing, Recording Tests, JUnit Sampler, Access Log Sampler, Extending JMeter), 'Community' (Issue Tracking, Security, Mailing Lists, Source Repositories, Building and Contributing, Project info at Apache, Contributors), and 'Foundation'. The main content area is titled 'Download Apache JMeter' and includes a warning about verifying the integrity of downloaded files. It provides a list of mirrors, with the current one being <https://d1cdn.apache.org/>. Below this, it explains the 'KEYS' link for code signing keys and the 'SHA-512' link for checksums. It also mentions the 'Apache JMeter' site for more information. The page is divided into sections for 'Binaries' and 'Source'. The 'Binaries' section lists two download links: [apache-jmeter-5.6.3.lgz sha512 pgp](#) and [apache-jmeter-5.6.3.zip sha512 pgp](#). The 'Source' section lists two download links: [apache-jmeter-5.6.3_src.lgz sha512 pgp](#) and [apache-jmeter-5.6.3_src.zip sha512 pgp](#). The page is in English and the date is 19-04-2026.

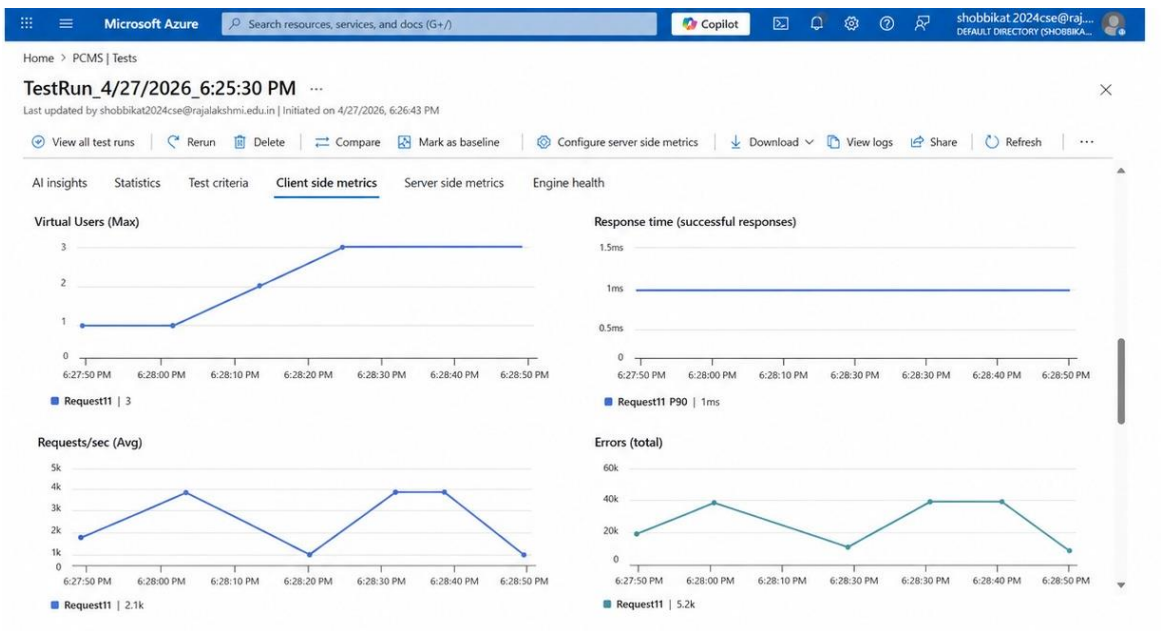
5. Test Plan



6. Creating the bug



7. Response Time Graph



8. Summary report

Azure DevOps shobhikat2024cse / PRODUCT CATEGORY MANAGEMENT SYSTEM / Pipelines

Connect Select Configure **Review**

New pipeline

Review your pipeline YAML

Variables Save and run

shobbika/PRODUCT-CATEGORY-MANAGEMENT-SYSTEM / azure-pipelines-1.yml

```

1 # Python to Linux Web App on Azure
2 # Build your Python project and deploy it to Azure as a Linux Web App.
3 # Change python version to one that's appropriate for your application.
4 # https://docs.microsoft.com/azure/devops/pipelines/languages/python
5
6 trigger:
7   - main
8
9 variables:
10  # Azure Resource Manager connection created during pipeline creation
11  azureServiceConnectionId: 'f8a56bfe-502c-484a-929f-cda157746e43'
12
13  # Web app name
14  webAppName: 'ProductCategoryManagementSystem'
15
16  # Agent VM image name
17  vmImageName: 'ubuntu-latest'
18
19  # Environment name
20  environmentName: 'ProductCategoryManagementSystem'

```

Microsoft Azure Search resources, services, and docs (G+/) Copilot

shobhikat.2024cse@raj... DEFAULT DIRECTORY (SHOBHIKA...)

Home > PCMS | Tests

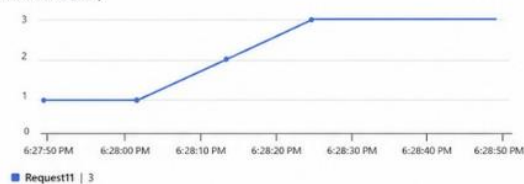
TestRun_4/27/2026_6:25:30 PM

Last updated by shobhikat2024cse@rajalakshmi.edu.in | Initiated on 4/27/2026, 6:26:43 PM

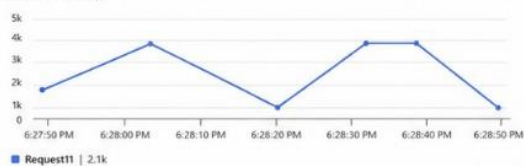
View all test runs Rerun Delete Compare Mark as baseline Configure server side metrics Download View logs Share Refresh

AI insights Statistics Test criteria **Client side metrics** Server side metrics Engine health

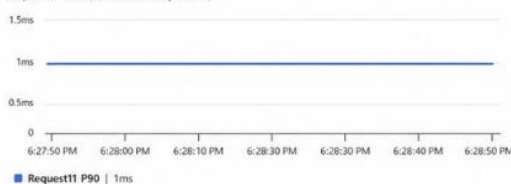
Virtual Users (Max)



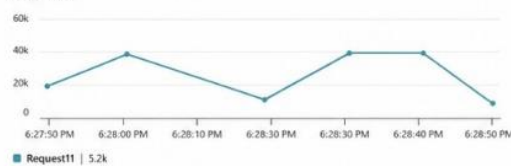
Requests/sec (Avg)



Response time (successful responses)



Errors (total)



Result:

The test plans and test cases for the user stories is created in Azure with Happy Path and Error Path

EXP NO: 09	LOAD TESTING AND CI/CD PIPELINE SETUP
------------	---------------------------------------

Aim:

To perform load testing for the Product Tracking System and to create and demonstrate a Azure-integrated CI/CD pipeline for automating application builds, tests, and deployment .

Load Testing

Load Testing:

Load Testing is used to simulate multiple users accessing the application simultaneously to evaluate system performance, response time, and stability under heavy traffic. It helps identify bottlenecks, and improve scalability before deployment.

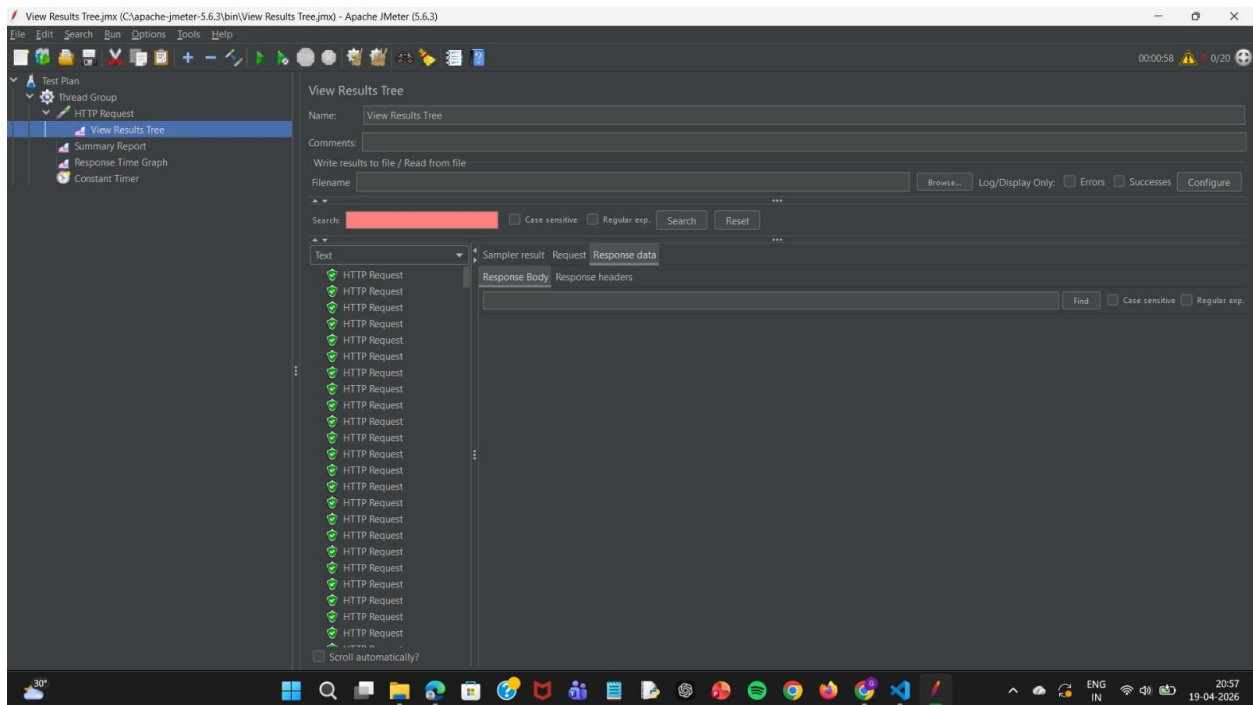
Steps to Perform Load Testing:

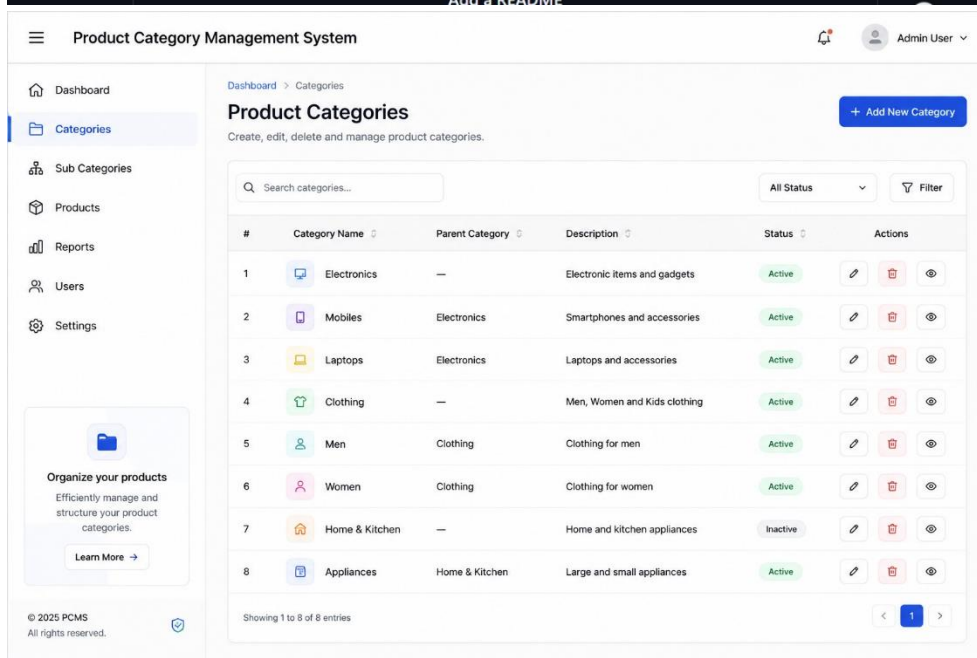
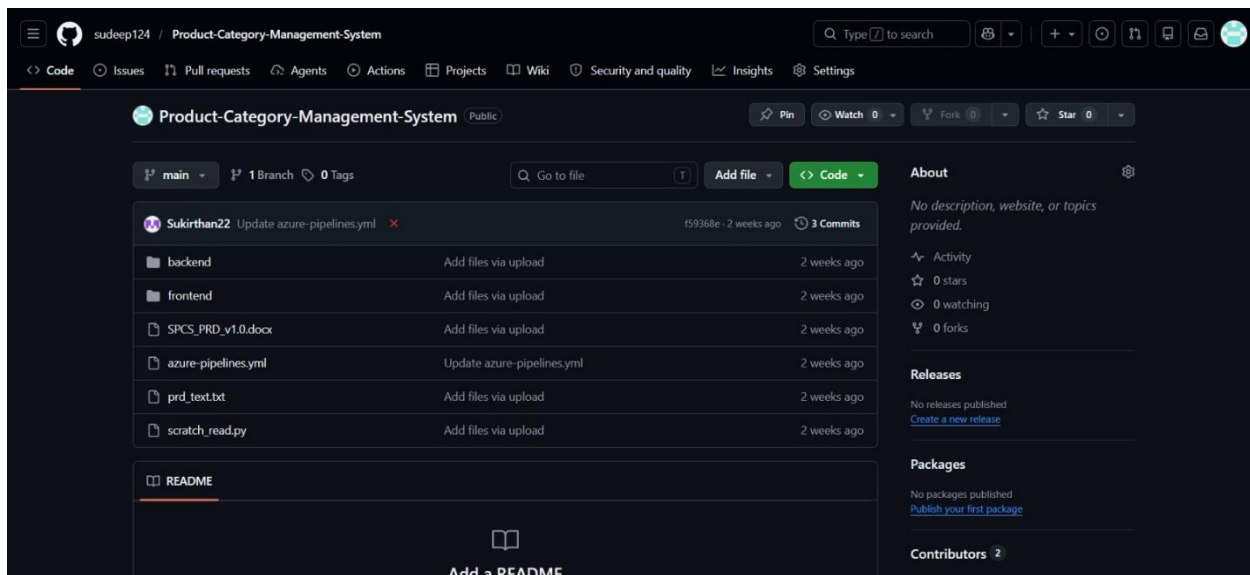
1. Sign in to the selected load testing tool (Apache JMeter / Locust / online load testing tool).
2. Create a New Test Plan
 - o Open the tool and create a new test plan.
3. Configure Thread Group
 - o **Number of Users:** Enter virtual users.
 - o **Ramp-Up Time:** Set user start time.
 - o **Loop Count:** Set number of requests.
4. Add HTTP Request
 - o Server Name: localhost / deployed server IP.
 - o Port Number: 8080
 - o Path: /track?productId=product1
5. Add Listeners
 - o View Results Tree
 - o Summary Report
 - o Graph Results
6. Save the test plan and click Start.
7. Observe performance metrics such as:
 - o Response time
 - o Throughput
 - o Error percentage
 - o CPU usage

Steps to Create and Demonstrate CI/CD Pipeline:

1. Open GitHub repository containing the Product Tracking System source code.
2. Open GitHub Actions / CI-CD tool.
3. Create a new workflow file.
4. Configure pipeline stages:
 - Build Java project using Maven
 - Run test cases
 - Package application
 - Deploy application
5. Save the workflow file.
6. Trigger the pipeline by pushing code changes.
7. Verify successful build, test, and deployment status.

Load Testing





Pipelines

Description:

This experiment demonstrates how to connect a GitHub-hosted Java-based Product Tracking System project with Azure. The pipeline automatically installs dependencies, builds the project using Maven, runs

tests, and publishes build results. This ensures every code commit is verified for reliability and smooth deployment.

Steps:

1. Connect GitHub Azure:
 - o Open Azure project settings.
 - o Select GitHub integration.
 - o Authorize access to your GitHub repository.
 - o Link the repository with the Azure project.
2. Create GitHub Actions Workflow:
 - o In your GitHub repository, open Actions tab.
 - o Create a new workflow file inside:
 .github/workflows/main.yml
3. Add Pipeline Configuration:
 - o Configure Java setup.
 - o Install Maven dependencies.
 - o Compile project.
 - o Run tests.
 - o Package application.
4. Commit and Push Code:
 - o Push changes to GitHub repository.
5. Automatic Pipeline Execution:
 - o GitHub Actions automatically starts pipeline after each push.
6. Verify Results:
 - o Check build success, test results, and deployment status.

Main.yml code

name: Java CI Pipeline

on:

push:

branches:

- main

jobs:

build:

runs-on: ubuntu-latest

steps:

Step 1: Checkout code from GitHub

- name: Checkout Repository

uses: actions/checkout@v4

```
# Step 2: Set up Java
- name: Set up JDK 17
  uses: actions/setup-java@v4
  with:
    java-version: '17'
    distribution: 'temurin'
```

```
# Step 3: Build using Maven
- name: Build Project
  run: mvn clean compile
```

```
# Step 4: Run Tests
- name: Run Tests
  run: mvn test
```

```
# Step 5: Package Application
- name: Package Project
  run: mvn package
```

```
# Step 6: Success Message
- name: Display Message
  run: echo "Product Tracking System Build Successful!"
```

7. Pipeline Tasks Include:

- Setting up the Java environment using JDK 17.
- Installing project dependencies and required libraries using Maven.
- Compiling the Product Tracking System source code using Maven build commands.
- Running test cases to verify application functionality.
- Packaging the project into a deployable JAR file.
- Displaying a success message after successful pipeline execution.

8. Run and Monitor Pipeline:

- Commit changes to the main branch of your GitHub repository to trigger the pipeline automatically.
- Open the GitHub Actions tab to monitor workflow execution.
- View logs, build status, test results, errors, or success messages and ensure the pipeline runs successfully.

Pipeline

The screenshot shows a GitHub Actions workflow run for the repository 'Simple-Online-Tracking-System'. The workflow is named 'Update maven.yml' and is part of a 'Java CI Pipeline'. The run was triggered by a push to the 'main' branch by user 'gowtham1156' and has a status of 'Success' with a total duration of 26 seconds. The workflow file 'maven.yml' is shown, and the 'build' job is highlighted, indicating it ran successfully in 22 seconds. An annotation at the bottom of the build job provides a warning about deprecated Node.js 20 actions, stating that they will be removed by default starting June 2nd, 2026, and suggesting the use of Node.js 24 or the 'FORCE_JAVASCRIPT_ACTIONS_TO_NODE24=true' setting.

CI/CD Pipeline Explanation x Update maven.yml - Gokulan00 x New tab x New tab x + Ask Gemini x

github.com/Gokulan00/Simple-Online-Tracking-System/actions/runs/24631925885

Gokulan00 / Simple-Online-Tracking-System

Type to search

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Java CI Pipeline

Update maven.yml #2 Re-run all jobs

Summary

All jobs

build

Run details

Usage

Workflow file

Triggered via push 3 minutes ago

Status Success

Total duration 26s

Artifacts -

gowtham1156 pushed -> 994e779 main

maven.yml

on: push

build 22s

Annotations

1 warning

build

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/setup-java@v4. Actions will be forced to run with Node.js 24 by default starting June 2nd, 2026. Node.js 20 will be removed from the runner on September 16th, 2026. Please check if updated versions of these actions are available that support Node.js 24. To opt into Node.js 24 now, set the FORCE_JAVASCRIPT_ACTIONS_TO_NODE24=true

10° Search

ENG IN 2030 19-04-2026

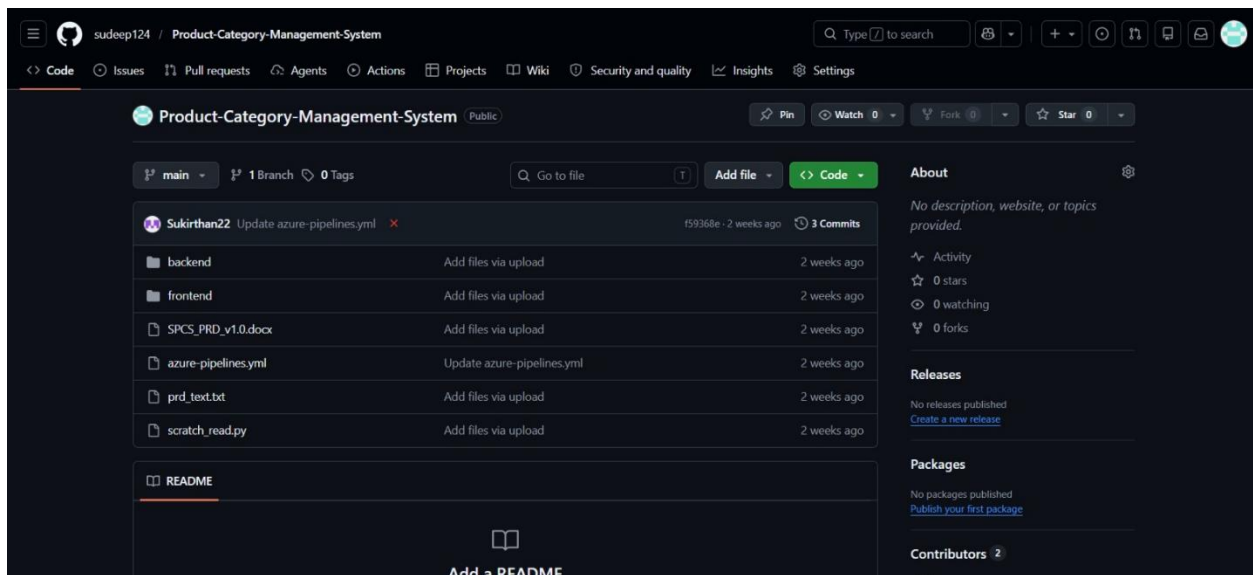
Result: Successfully created the Load Testing resource and executed a load test to assess the performance of the specified endpoint and also demonstrated pipelines in Azure.

EXP NO: 10	GITHUB: PROJECT STRUCTURE & NAMING CONVENTIONS
------------	---

Aim:

To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the Product Tracking System project.

GitHub Project Structure



Result:

The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the codebase.